

Sir Syed University of Engineering & Technology
Computer Engineering Department
COMPUTER ORGANIZATION AND ARCHITECTURE (CE-207L)

Semester: 4th
Batch: Fall 2024
Section: A

Project Report
EXPENSE TRACKER



Submitted by:

Group Member	Roll Number
M. Ayan Azhar	2024F-BCE-011
Shahzeb Shahid	2024F-BCE-029

Submitted To:

Miss Engr. Aaima Sohail

Acknowledgement

We would like to express our sincere gratitude to everyone who contributed to the successful completion of our project, "**FinASM Expense Tracker**."

First and foremost, we are deeply thankful to our parents for their continuous support, encouragement, and motivation throughout the development of this project. Their confidence in us inspired us to overcome challenges and achieve our goals.

We would like to extend our heartfelt appreciation to our respected teacher, **Miss Engr. Aaima Sohail**, for her valuable guidance, constructive feedback, and continuous encouragement. Her knowledge and support played a vital role in the successful completion of this project.

Finally, we thank everyone who directly or indirectly contributed to the completion of the **FinASM Expense Tracker** project. Their support and encouragement have been greatly appreciated.

M. Ayan Azhar
Shahzeb Shahid

S.No Contents		Page No
1.	INTRODUCTION	Page 4
2.	METHODOLOGY	Page 4
3.	PROJECT DESCRIPTION	Page 5
4.	KEY FEATURES	Page 6
5.	SOFTWARE USE	Page 7
6.	SOURCE CODE	Page 7
7.	OUTPUT	Page 21
8.	CONCLUSION	Page 23

INTRODUCTION

Managing personal finances is an important part of everyday life. Many people find it difficult to track their income and expenses manually. To solve this problem, the **FinASM Expense Tracker** was developed. FinASM is a finance management system that allows users to record income and expenses, monitor their balance, and view financial summaries through a user-friendly interface.

The project combines **MIPS Assembly Language** and **Python GUI programming** to demonstrate both low-level and high-level software development concepts. It provides an educational and practical solution for financial tracking.

METHODOLOGY

The project was developed using the following methodology:

1. Requirement Analysis

- Identify the need for tracking income and expenses.
- Determine required categories and financial reports.

2. System Design

- Design data structures for storing records.
- Create a menu-driven assembly program.
- Design a graphical user interface using Python.

3. Implementation

- Develop the core expense tracking logic in MIPS Assembly.
- Develop the graphical interface using Tkinter.
- Integrate data visualization and report generation features.

4. Testing

- Verify income and expense calculations.
- Test record deletion and dashboard updates.
- Validate PDF report generation and pie chart visualization.

5. Evaluation

- Ensure accuracy of financial calculations.
- Verify user-friendly operation and system stability.

PROJECT DESCRIPTION

FinASM Expense Tracker is a personal finance management application designed to help users manage their financial activities efficiently.

The system allows users to:

- Add income records.
- Add expense records.
- Categorize financial transactions.
- Monitor available balance.
- View dashboard statistics.
- Generate PDF reports.
- Analyze spending patterns using pie charts.

The project consists of two major components:

1. MIPS Assembly Module

The assembly program provides:

- Menu-driven interface.
- Income and expense entry.
- Category selection.
- Transaction storage.
- Summary table generation.
- Balance calculation.

2. Python GUI Module

The graphical interface provides:

- Interactive dashboard.
- Record management.
- Financial statistics.
- Pie chart visualization.
- PDF report generation.
- Light and dark theme modes.

KEY FEATURES

Financial Management

- Income tracking
- Expense tracking
- Real-time balance calculation
- Multiple transaction categories

Dashboard

- Total income display
- Total expense display
- Savings calculation
- Transaction summary table

Data Visualization

- Expense breakdown pie chart
- Category-wise spending analysis

User Interface

- User-friendly GUI
- Modes (Light, Dark)

Reporting

- PDF report generation
- Transaction history records

Data Management

- Add records
- Delete records
- Automatic balance updates
- Save records to file

SOFTWARE USE

The following software and technologies were used in the development of this project:

Software / Technology	Purpose
MIPS Assembly Language	Core financial processing
MARS Simulator	Execution of Assembly code
Python	GUI development
Tkinter	Graphical User Interface
Matplotlib	Pie chart visualization
ReportLab	PDF report generation
Text File Storage	Data persistence
Windows OS	Development environment

SOURCE CODE

The source code consists of two major files:

finasm.asm

```
1 .data
2 welcome: .asciiz "\n--- FinASM: MIPS Expense Tracker ---\n"
3 menu: .asciiz "1. Add Income\n2. Add Expense\n3. View Summary Table\n4. Exit\nChoice: "
4 inc_cat_menu: .asciiz "\nIncome Category:\n1. Salary\n2. Business\n3. Rental Income\n4.
Government Benefits\n5. Commissions\n6. Other Income\nChoice: "
5 exp_cat_menu: .asciiz "\nExpense Category:\n1. Food\n2. Savings & Investments\n3. Transport\n4.
Medical\n5. Home Insurance\n6. Personal & Family\n7. Entertainment\n8. Other\nChoice: "
6 inc_msg: .asciiz "Enter income amount: "
7 exp_msg: .asciiz "Enter expense amount: "
8
9 table_hdr: .asciiz "\nType | Category | Amount\n-----\n"
10 mes: .asciiz "\nGenerating Summary...\n"
11 newline: .asciiz "\n"
12 inc_lbl: .asciiz " [I] | "
13 exp_lbl: .asciiz " [E] | "
14
15 # Income Category Names
16 ic1: .asciiz "Salary | Rs"
17 ic2: .asciiz "Business | Rs"
18 ic3: .asciiz "Rental Income | Rs"
19 ic4: .asciiz "Govt Benefits | Rs"
20 ic5: .asciiz "Commissions | Rs"
21 ic6: .asciiz "Other Income | Rs"
22
23 # Expense Category Names
24 ec1: .asciiz "Food | Rs"
25 ec2: .asciiz "Savings | Rs"
26 ec3: .asciiz "Transport | Rs"
27 ec4: .asciiz "Medical | Rs"
28 ec5: .asciiz "Home Insurance | Rs"
29 ec6: .asciiz "Personal & Family | Rs"
30 ec7: .asciiz "Entertainment | Rs"
31 ec8: .asciiz "Other Expense | Rs"
32 total_lbl: .asciiz "Total Balance | Rs"
```

```

34  # Storage (Max 50)
35
36  history_amt: .word 0:50
37  history_typ: .byte 0:50
38  history_cat: .word 0:50
39
40  .text
41  .globl main
42  main:
43      li $s0, 0          # Total Balance
44      li $s1, 0          # Counter
45
46  menu_loop:
47      li $v0, 4
48      la $a0, welcome
49      syscall
50      la $a0, menu
51      syscall
52
53      li $v0, 5
54      syscall
55      move $t0, $v0
56
57      beq $t0, 1, add_income
58      beq $t0, 2, add_expense
59      beq $t0, 3, show_table
60      beq $t0, 4, exit_program
61      j menu_loop
62
63  add_income:
64      li $v0, 4
65      la $a0, inc_msg
66      syscall
67      li $v0, 5
68      syscall
69      move $t1, $v0
70      add $s0, $s0, $t1
71
72      li $v0, 4
73      la $a0, inc_cat_menu
74      syscall
75      li $v0, 5
76      syscall
77      move $t8, $v0
78
79      li $t2, 'I'
80      jal store_data
81      j menu_loop
82
83  add_expense:
84      li $v0, 4
85      la $a0, exp_msg
86      syscall
87      li $v0, 5
88      syscall
89      move $t1, $v0
90      sub $s0, $s0, $t1
91
92      li $v0, 4
93      la $a0, exp_cat_menu
94      syscall
95      li $v0, 5
96      syscall
97      move $t8, $v0

```



```

99     li $t2, 'E'
100    jal store_data
101    j menu_loop
102
103 store_data:
104     li $t3, 50
105     beq $s1, $t3, full
106     sll $t4, $s1, 2
107     sw $t1, history_amt($t4)
108     sw $t8, history_cat($t4)
109     sb $t2, history_typ($s1)
110     addi $s1, $s1, 1
111 full:
112     jr $ra
113
114 show_table:
115     li $v0, 4
116     la $a0, mes
117     syscall
118     la $a0, table_hdr
119     syscall
120     li $t5, 0
121
122 table_loop:
123     beq $t5, $s1, total
124
125     lb $t6, history_typ($t5)
126     sll $t7, $t5, 2
127     lw $t9, history_cat($t7)
128
129     li $v0, 4
130     beq $t6, 'I', print_inc_row
131
132 # Print Expense Row
133 la $a0, exp_lbl
134 syscall
135 beq $t9, 1, e1
136 beq $t9, 2, e2
137 beq $t9, 3, e3
138 beq $t9, 4, e4
139 beq $t9, 5, e5
140 beq $t9, 6, e6
141 beq $t9, 7, e7
142 la $a0, ec8      # Default to Other Expense
143 j print_val
144
145 e1: la $a0, ec1
146     j print_val
147 e2: la $a0, ec2
148     j print_val
149 e3: la $a0, ec3
150     j print_val
151 e4: la $a0, ec4
152     j print_val
153 e5: la $a0, ec5
154     j print_val
155 e6: la $a0, ec6
156     j print_val
157 e7: la $a0, ec7
158     j print_val
159
160 print_inc_row:
161 la $a0, inc_lbl
162 syscall
163 beq $t9, 1, i1
164 beq $t9, 2, i2
165 beq $t9, 3, i3
166 beq $t9, 4, i4
167 beq $t9, 5, i5
168 la $a0, ic6      # Default to Other Income
169 j print_val

```

```

171 i1: la $a0, ic1
172 j print_val
173 i2: la $a0, ic2
174 j print_val
175 i3: la $a0, ic3
176 j print_val
177 i4: la $a0, ic4
178 j print_val
179 i5: la $a0, ic5
180 j print_val
181
182 print_val:
183 syscall
184 lw $a0, history_amt($t7)
185 li $v0, 1
186 syscall
187
188 li $v0, 4
189 la $a0, newline
190 syscall
191 addi $t5, $t5, 1
192 j table_loop
193
194 total:
195 li $v0, 4
196 la $a0, total_lbl
197 syscall
198
199 move $a0, $s0 # Move net balance to $a0
200 li $v0, 1 # Print balance
201 syscall
202
203 li $v0, 4
204 la $a0, newline
205 syscall
206 syscall # Double space for neatness
207 j menu_loop
208
209 exit_program:
210 li $v0, 10
211 syscall

```

gui.py

```

1 from tkinter import *
2 from tkinter import ttk, messagebox
3 from reportlab.pdfgen import canvas
4
5 # PIE CHART
6
7 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
8 import matplotlib.pyplot as plt
9
10
11 ASM_FILE = "finasm.asm"
12
13
14 MARS_PATH = r"C:\Users\Yousuf Traders\OneDrive\Desktop\Mars4_5.jar"
15
16
17 records = []
18 balance = 0
19 current_mode = "dark"
20
21 # ===== WINDOW =====
22
23 root = Tk()
24 root.title("FinASM Expense Tracker")
25 root.geometry("1350x760")
26 root.configure(bg="#1e1e1e")

```

```

28 # ===== THEME FUNCTION =====
29
30 def set_mode(mode):
31
32     global current_mode
33
34     current_mode = mode
35
36     if mode == "light":
37
38         bg_main = "white"
39         text_main = "black"
40         panel_bg = "#f0f0f0"
41
42     else:
43
44         bg_main = "#1e1e1e"
45         text_main = "white"
46         panel_bg = "#2d2d2d"
47
48     root.configure(bg=bg_main)
49
50     title.config(bg=bg_main, fg=text_main)
51
52     input_panel.config(bg=bg_main)
53
54     left_panel.config(bg=panel_bg)
55
56     right_panel.config(bg=bg_main)
57
58     balance_label.config(
59         bg=bg_main,
60         fg="green" if mode == "light" else "lightgreen"
61     )
62
63     warning_label.config(
64         bg=bg_main
65     )
66
67     dashboard_panel.config(bg=bg_main)
68
69     top_frame.config(bg=bg_main)
70
71     middle_frame.config(bg=bg_main)
72
73     table_frame.config(bg=bg_main)
74
75     chart_frame.config(bg=bg_main)
76
77     bottom_frame.config(bg=bg_main)
78
79     category_label.config(bg=panel_bg, fg=text_main)
80
81     amount_label.config(bg=panel_bg, fg=text_main)
82
83     show_pie_chart()
84

```

```

85 # ===== FUNCTIONS =====
86
87 def update_balance():
88     balance_label.config(text=f"Balance: Rs {balance}")
89
90 def save_data():
91     with open("data.txt", "w") as file:
92         for rec in records:
93             line = f"{rec[0]},{rec[1]},{rec[2]}\n"
94             file.write(line)
95
96 # ===== DELETE FUNCTION =====
97
98 def delete_record():
99     global balance
100     selected_item = tree.selection()
101     if not selected_item:
102         messagebox.showerror("Error", "Select Record First")
103         return
104     item = selected_item[0]
105     values = tree.item(item, "values")
106     rec_type = values[0]
107     category = values[1]
108     amount = int(values[2])
109
110     # REMOVE FROM RECORDS
111     for rec in records:
112         if rec[0] == rec_type and rec[1] == category and rec[2] == amount:
113             records.remove(rec)
114             break
115
116     # UPDATE BALANCE
117     if rec_type == "Income":
118         balance -= amount
119     else:
120         balance += amount
121
122     tree.delete(item)

```

```

135     # REMOVE WARNING IF BALANCE POSITIVE
136     if balance > 0:
137         warning_label.config(text="")
138
139     update_balance()
140
141     update_dashboard()
142
143     refresh_dashboard_table()
144
145     save_data()
146
147     messagebox.showinfo("Deleted", "Record Deleted Successfully")
148
149 # ===== ADD INCOME =====
150
151 def add_income():
152
153     global balance
154
155     category = category_box.get()
156     amount = amount_entry.get()
157
158     if amount == "":
159         messagebox.showerror("Error", "Enter Amount")
160         return
161
162     amount = int(amount)
163
164     records.append(("Income", category, amount))
165
166     tree.insert("", END, values=("Income", category, amount))
167
168     balance += amount
169
170     # CLEAR WARNING
171     if balance > 0:
172         warning_label.config(text="")
173
174     update_balance()
175
176     update_dashboard()
177
178     refresh_dashboard_table()
179
180     save_data()
181
182     amount_entry.delete(0, END)
183
184 # ===== ADD EXPENSE =====
185
186 def add_expense():
187
188     global balance
189
190     category = category_box.get()
191     amount = amount_entry.get()
192
193     if amount == "":
194         messagebox.showerror("Error", "Enter Amount")
195         return
196
197     amount = int(amount)
198

```

```

199 # ===== NEGATIVE CHECK =====
200
201 if balance - amount < 0:
202     messagebox.showerror(
203         "Over Budget",
204         "You are over budget!\nExpense cannot be added."
205     )
206
207     warning_label.config(
208         text="⚠ You Are Over Budget!",
209         fg="red"
210     )
211
212     return
213
214 # ===== ADD RECORD =====
215
216 records.append(("Expense", category, amount))
217
218 tree.insert("", END, values=("Expense", category, amount))
219
220 balance -= amount
221
222 # ===== ZERO BALANCE WARNING =====
223
224 if balance == 0:
225     messagebox.showwarning(
226         "Warning",
227         "Balance is 0.\nYou are over budget!"
228     )
229
230     warning_label.config(
231         text="⚠ Balance is 0 - You Are Over Budget!",
232         fg="red"
233     )
234
235 else:
236     warning_label.config(text="")
237
238 update_balance()
239
240 update_dashboard()
241
242 refresh_dashboard_table()
243
244 save_data()
245
246 amount_entry.delete(0, END)
247
248
249

```

```

251 # ===== PDF =====
252
253 def generate_pdf():
254
255     pdf = canvas.Canvas("Expense_Report.pdf")
256
257     pdf.setFont("Helvetica-Bold", 20)
258
259     pdf.drawString(180, 800, "FinASM Expense Report")
260
261     y = 740
262
263     for rec in records:
264
265         text = f"{rec[0]} | {rec[1]} | Rs {rec[2]}"
266
267         pdf.drawString(80, y, text)
268
269         y -= 25
270
271     pdf.drawString(80, y - 30, f"Final Balance: Rs {balance}")
272
273     pdf.save()
274
275     messagebox.showinfo("Success", "PDF Generated Successfully")
276
277 # ===== DASHBOARD =====
278
279 def show_dashboard():
280
281     input_panel.pack_forget()
282
283     refresh_dashboard_table()
284
285     dashboard_panel.pack(fill=BOTH, expand=True)
286
287 def back_to_input():
288
289     dashboard_panel.pack_forget()
290
291     input_panel.pack(fill=BOTH, expand=True)
292
293 def update_dashboard():
294
295     total_income = 0
296     total_expense = 0
297
298     for rec in records:
299
300         if rec[0] == "Income":
301             total_income += rec[2]
302         else:
303             total_expense += rec[2]
304
305     savings = total_income - total_expense
306
307     income_value.config(text=f"Rs {total_income}")
308
309     expense_value.config(text=f"Rs {total_expense}")
310
311     savings_value.config(text=f"Rs {savings}")
312
313     show_pie_chart()

```

```

315 # ===== PIE CHART =====
316
317 def show_pie_chart():
318
319     expense_data = {}
320
321     for rec in records:
322
323         if rec[0] == "Expense":
324
325             category = rec[1]
326             amount = rec[2]
327
328             if category in expense_data:
329                 expense_data[category] += amount
330             else:
331                 expense_data[category] = amount
332
333     # REMOVE OLD CHART
334     for widget in chart_frame.wininfo_children():
335         widget.destroy()
336
337     if len(expense_data) == 0:
338         return
339
340     labels = list(expense_data.keys())
341
342     amounts = list(expense_data.values())
343
344     colors = [ "#ff6666", "#3399ff", "#ffcc4d", "#4dd2cc", "#9966ff",
345               "#66ff99", "#ff99cc", "#ff5e5e" ]
346
347     explode = [0.02] * len(amounts)
348
349     fig, ax = plt.subplots(figsize=(9, 5))
350
351     # MODE COLORS
352     if current_mode == "dark":
353         bg_color = "#000000"
354         text_color = "white"
355     else:
356         bg_color = "white"
357         text_color = "black"
358     fig.patch.set_facecolor(bg_color)
359     ax.set_facecolor(bg_color)
360
361     # ===== SHOW ALL PERCENTAGES =====
362     def format_pct(pct):
363         return f"{pct:.1f}%"
364
365     wedges, texts, autotexts = ax.pie(
366         amounts,
367         colors=colors,
368         explode=explode,
369         startangle=90,
370         autopct=format_pct,
371         pctdistance=0.82,
372         labeldistance=1.08,
373         textprops={
374             'color': text_color,
375             'fontsize': 11,
376             'weight': 'bold'
377         }
378     )
379
380     # ===== TEXT COLORS =====
381     for text in texts:
382         text.set_color(text_color)
383     for autotext in autotexts:
384         autotext.set_color(text_color)

```



```

383 # ===== TITLE =====
384 ax.set_title(
385     "Expense Breakdown (Pie Chart)",
386     fontsize=18,fontweight="bold",
387     color=text_color,pad=20
388 )
389 # ===== LEGEND =====
390 legend = ax.legend(
391     wedges,labels,title="Categories",loc="center left",
392     bbox_to_anchor=(1.0, 0.5),fontsize=11,frameon=False
393 )
394 legend.get_title().set_color(text_color)
395 for text in legend.get_texts():
396     text.set_color(text_color)
397 ax.axis('equal')
398 plt.subplots_adjust( left=0.05, right=0.78,top=0.90,
399     bottom=0.05)
400 chart = FigureCanvasTkAgg(fig, chart_frame)
401 chart.draw()
402 chart.get_tk_widget().pack(fill=BOTH, expand=True)
403 # ===== TITLE =====
404
405 title = Label(root,text="FinASM Expense Tracker",
406     font=("Arial", 24, "bold"),bg="#1e1e1e",fg="white")
407 title.pack(pady=10)
408
409 # =====
410 # INPUT PANEL
411 # =====
412
413 input_panel = Frame(root, bg="#1e1e1e")
414
415 input_panel.pack(fill=BOTH, expand=True)
416
417 # ===== LEFT PANEL =====
418 left_panel = Frame(input_panel, bg="#2d2d2d", width=250)
419 left_panel.pack(side=LEFT, fill=Y)
420 # ===== RIGHT PANEL =====
421 right_panel = Frame(input_panel, bg="#1e1e1e")
422 right_panel.pack(side=RIGHT, fill=BOTH, expand=True)
423 # ===== TABLE =====
424 columns = ("Type", "Category", "Amount")
425 tree = ttk.Treeview(right_panel,columns=columns,show="headings",
426     height=16)
427 for col in columns:
428     tree.heading(col, text=col)
429     tree.column(col, width=180, anchor=CENTER)
430 tree.pack(pady=20)
431 # ===== BALANCE =====
432 balance_label = Label( right_panel,text="Balance: Rs 0",
433     font=("Arial", 18, "bold"),bg="#1e1e1e",fg="lightgreen")
434 balance_label.pack()

```

```

436 # ===== WARNING LABEL =====
437 warning_label = Label(
438     right_panel, text="", font=("Arial", 14, "bold"),
439     bg="#1e1e1e", fg="red")
440 warning_label.pack(pady=10)
441 # ===== CATEGORY =====
442 category_label = Label(
443     left_panel, text="Category", font=("Arial", 12),
444     bg="#2d2d2d", fg="white")
445 category_label.pack(pady=10)
446 categories = [
447     "Salary",
448     "Business",
449     "Rental Income",
450     "Government Benefits",
451     "Commissions",
452     "Other Income",
453     "Food",
454     "Savings",
455     "Transport",
456     "Medical",
457     "Home Insurance",
458     "Personal & Family",
459     "Entertainment",
460     "Other Expense"
461 ]
462
463 category_box = ttk.Combobox(left_panel, values=categories, width=25)
464 category_box.pack(pady=5)
465 # ===== AMOUNT =====
466 amount_label = Label(
467     left_panel, text="Amount", font=("Arial", 12), bg="#2d2d2d",
468     fg="white")
469
470 amount_label.pack(pady=10)
471 amount_entry = Entry(left_panel, font=("Arial", 12))
472 amount_entry.pack(pady=5)
473 # ===== BUTTONS =====
474 Button(
475     left_panel, text="Add Income", command=add_income, bg="green",
476     fg="white", width=20, height=2).pack(pady=8)
477 Button(
478     left_panel, text="Add Expense", command=add_expense, bg="red",
479     fg="white", width=20, height=2).pack(pady=8)
480 Button(
481     left_panel, text="Delete Selected Record", command=delete_record,
482     bg="darkred", fg="white", width=20, height=2).pack(pady=8)
483 Button(
484     left_panel, text="Result Dashboard", command=show_dashboard,
485     bg="orange", fg="white", width=20, height=2).pack(pady=8)

```

```

487 Button(
488     left_panel, text="Light Mode", command=lambda: set_mode("light"),
489     bg="white", fg="black", width=20, height=2).pack(pady=8)
490
491 Button(
492     left_panel, text="Dark Mode", command=lambda: set_mode("dark"),
493     bg="black", fg="white", width=20, height=2).pack(pady=8)
494 # DASHBOARD PANEL
495 dashboard_panel = Frame(root, bg="#000000")
496 dashboard_panel.pack_propagate(False)
497 # ===== TOP BOXES =====
498 top_frame = Frame(dashboard_panel, bg="#000000")
499 top_frame.pack(pady=10)
500 # ===== INCOME BOX =====
501 income_box = Frame(top_frame, bg="#d4edda", bd=2, relief=RIDGE)
502 income_box.grid(row=0, column=0, padx=20)
503 Label(
504     income_box, text="Total Income", font=("Arial", 18, "bold"),
505     bg="#d4edda", fg="green").pack(padx=60, pady=10)
506
507 income_value = Label(
508     income_box, text="Rs 0", font=("Arial", 24, "bold"),
509     bg="#d4edda", fg="green")
510 income_value.pack(pady=20)
511
512 # ===== EXPENSE BOX =====
513 expense_box = Frame(top_frame, bg="#f8d7da", bd=2, relief=RIDGE)
514 expense_box.grid(row=0, column=1, padx=20)
515 Label(
516     expense_box, text="Total Expense", font=("Arial", 18, "bold"),
517     bg="#f8d7da", fg="red").pack(padx=60, pady=10)
518 expense_value = Label(
519     expense_box, text="Rs 0", font=("Arial", 24, "bold"),
520     bg="#f8d7da", fg="red")
521 expense_value.pack(pady=20)
522 # ===== SAVINGS BOX =====
523 savings_box = Frame(top_frame, bg="#d1ecf1", bd=2, relief=RIDGE)
524 savings_box.grid(row=0, column=2, padx=20)
525 Label(
526     savings_box, text="Total Savings", font=("Arial", 18, "bold"),
527     bg="#d1ecf1", fg="blue").pack(padx=60, pady=10)
528
529 savings_value = Label(
530     savings_box, text="Rs 0", font=("Arial", 24, "bold"),
531     bg="#d1ecf1", fg="blue")
532 savings_value.pack(pady=20)

```

```

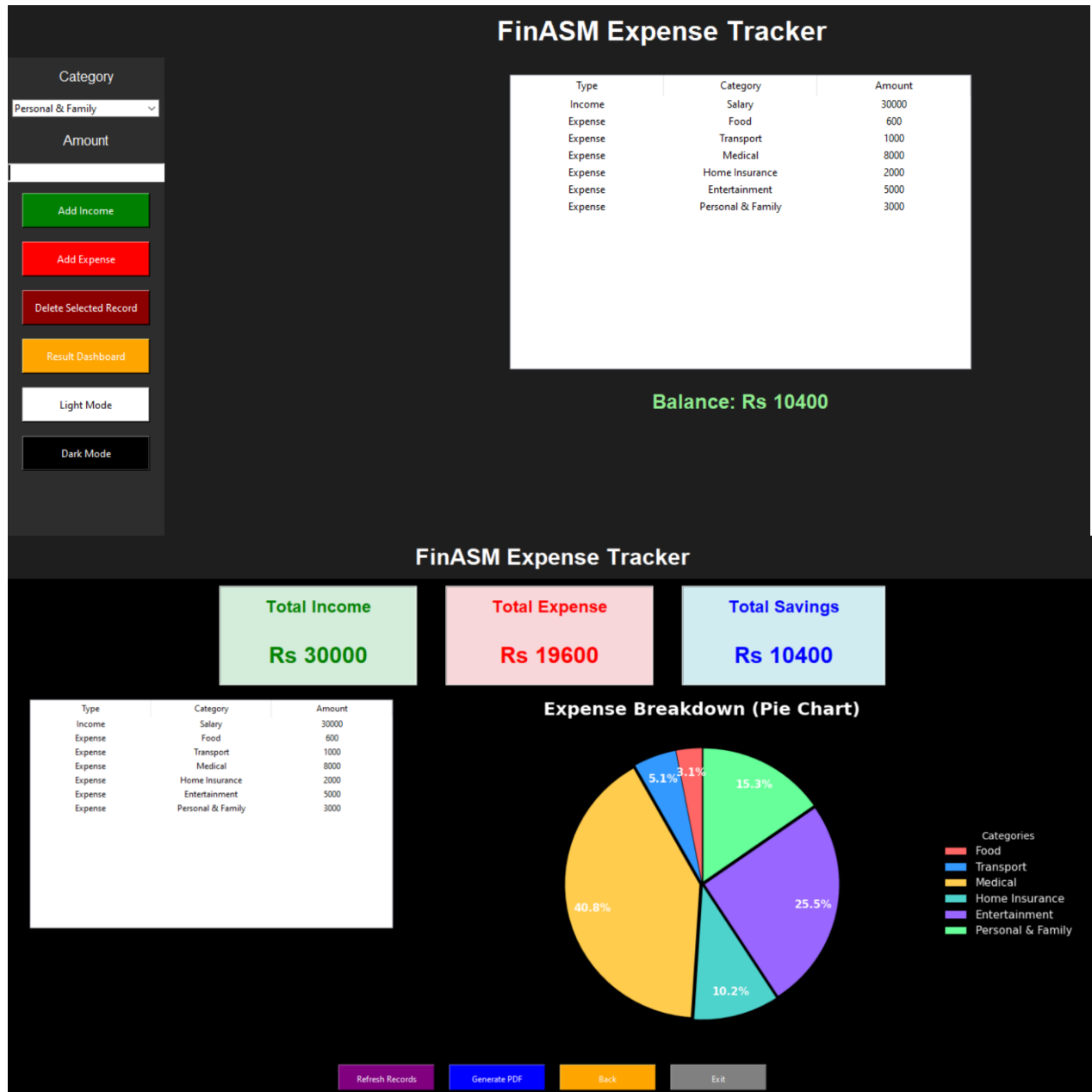
534 # TABLE + PIE CHART
535 middle_frame = Frame(dashboard_panel, bg="#000000")
536 middle_frame.pack(fill=BOTH, expand=True, pady=10)
537 middle_frame.columnconfigure(0, weight=1)
538 middle_frame.columnconfigure(1, weight=1)
539 # ===== TABLE FRAME =====
540 table_frame = Frame(middle_frame, bg="#000000")
541 table_frame.grid(row=0, column=0, padx=20, sticky="n")
542 # ===== RECORD TABLE =====
543 dashboard_table = ttk.Treeview(
544     table_frame, columns=columns, show="headings", height=15)
545 for col in columns:
546     dashboard_table.heading(col, text=col)
547     dashboard_table.column(col, width=170, anchor=CENTER)
548 dashboard_table.pack()

550 # ===== PIE CHART FRAME =====
551 chart_frame = Frame(middle_frame, bg="#000000")
552 chart_frame.grid(row=0, column=1, padx=20, sticky="n")
553 # ===== REFRESH FUNCTION =====
554 def refresh_dashboard_table():
555     for item in dashboard_table.get_children():
556         dashboard_table.delete(item)
557     for rec in records:
558         dashboard_table.insert("", END, values=rec)
559     show_pie_chart()
560 # BOTTOM BUTTON FRAME
561 bottom_frame = Frame(dashboard_panel, bg="#000000")
562 bottom_frame.pack(side=TOP, pady=10)
563 Button(
564     bottom_frame, text="Refresh Records", command=refresh_dashboard_table,
565     bg="purple", fg="white", width=18, height=2
566 ).grid(row=0, column=0, padx=10)
567 Button(
568     bottom_frame, text="Generate PDF", command=generate_pdf, bg="blue",
569     fg="white", width=18, height=2
570 ).grid(row=0, column=1, padx=10)
571 Button(
572     bottom_frame, text="Back", command=back_to_input, bg="orange",
573     fg="white", width=18, height=2
574 ).grid(row=0, column=2, padx=10)
575
576 Button(
577     bottom_frame, text="Exit", command=root.quit, bg="gray",
578     fg="white", width=18, height=2
579 ).grid(row=0, column=3, padx=10)
580 # ===== RUN =====
581 root.mainloop()

```

OUTPUT

Dark Mode:



Light Mode:

Category

Personal & Family

Amount

Add Income

Add Expense

Delete Selected Record

Result Dashboard

Light Mode

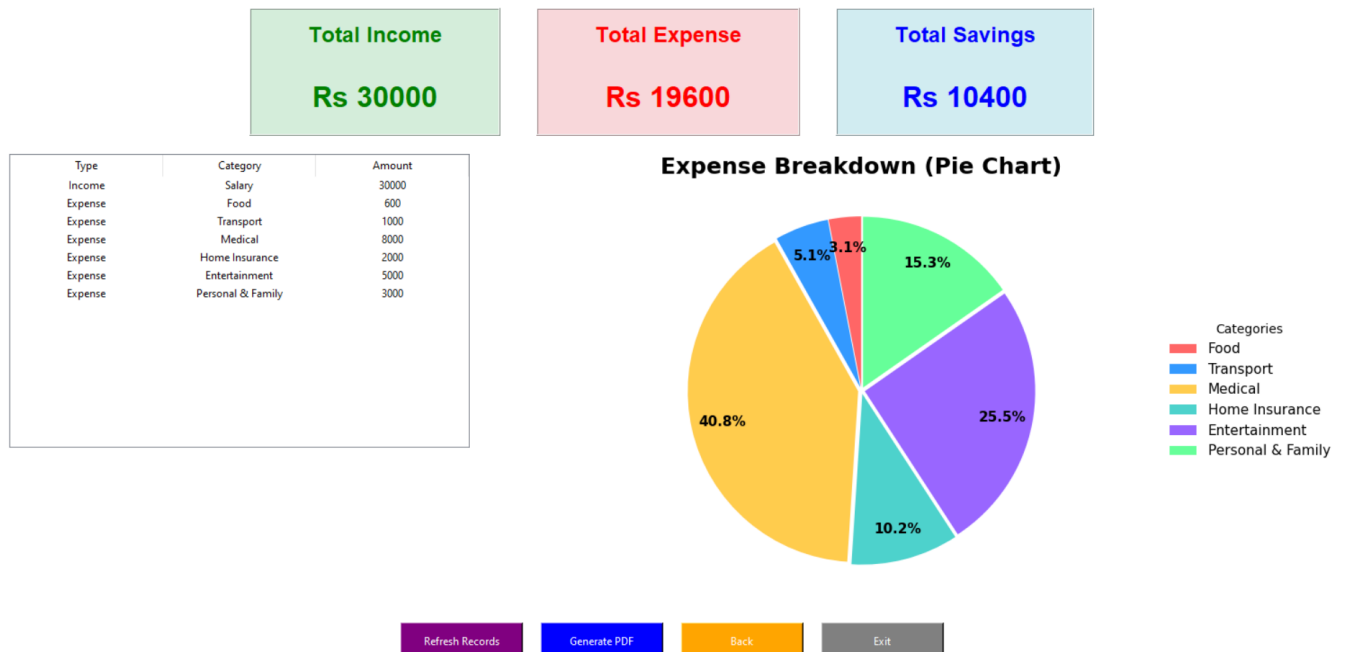
Dark Mode

FinASM Expense Tracker

Type	Category	Amount
Income	Salary	30000
Expense	Food	600
Expense	Transport	1000
Expense	Medical	8000
Expense	Home Insurance	2000
Expense	Entertainment	5000
Expense	Personal & Family	3000

Balance: Rs 10400

FinASM Expense Tracker



Type	Category	Amount
Income	Salary	30000
Expense	Food	600
Expense	Transport	1000
Expense	Medical	8000
Expense	Home Insurance	2000
Expense	Entertainment	5000
Expense	Personal & Family	3000

Expense Breakdown (Pie Chart)

Categories	Percentage
Food	3.1%
Transport	5.1%
Medical	40.8%
Home Insurance	10.2%
Entertainment	25.5%
Personal & Family	15.3%

Refresh Records

Generate PDF

Back

Exit

PDF Report

FinASM Expense Report

Income | Salary | Rs 30000

Expense | Food | Rs 600

Expense | Transport | Rs 1000

Expense | Medical | Rs 8000

Expense | Home Insurance | Rs 2000

Expense | Entertainment | Rs 5000

Expense | Personal & Family | Rs 3000

Final Balance: Rs 10400

Conclusion

The FinASM Expense Tracker successfully demonstrates the integration of MIPS Assembly Language with Python GUI programming. The project provides an effective solution for personal finance management by allowing users to record transactions, monitor balances, analyze spending patterns, and generate reports.

The system enhances financial awareness, improves budgeting practices, and serves as an educational example of combining low-level and high-level programming technologies in a single application. Future improvements may include database integration, user authentication, cloud storage, and advanced financial analytics.